

Java Programming

Mini Project Report On

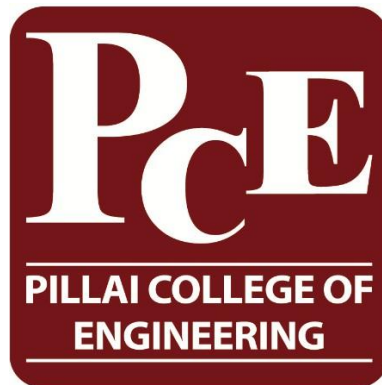
ONLINE COURSE ENROLLMENT SYSTEM

Submitted in partial fulfilment of the requirement for
the Degree of

**Bachelor of Technology in
Electronics & Telecommunication Engineering**

Submitted By:

Janhavi Nilesh Unde – EXTCA566



Supervisor:

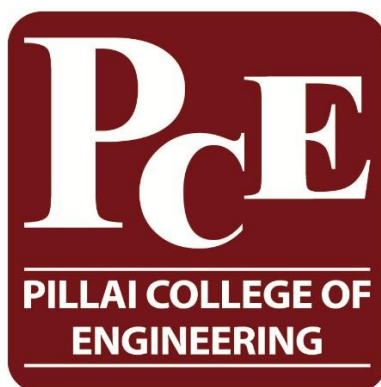
Dr. Aboli Khedkar

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION

PILLAI COLLEGE OF ENGINEERING

New Panvel – 410206

Academic Year 2026 - 27

**DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION**

Pillai College of Engineering New
Panvel – 410206

CERTIFICATE

This is to certify that the requirements for the Java Programming mini-project entitled '**Online Course Enrollment System**' have been successfully completed by the following students:

Student Name	Roll No.
Janhavi Nilesh Unde	EXTCA566

in partial fulfillment of **Bachelor of Technology in Electronics & Telecommunication Engineering** at Pillai College of Engineering, New Panvel – 410206 during the Academic Year 2026 – 2027.

Dr. Aboli Khedkar
Project Supervisor

Head of Department
Dept. of Electronics & Telecommunication Engineering

ABSTRACT

The **Online Course Enrollment System** is a desktop-native application developed in **Java** designed to automate course offerings, user authentication, and student registrations.

Designed around core

Object-Oriented Programming (OOP) principles—including inheritance, abstraction, encapsulation, and polymorphism—the application manages distinct user roles (**Student** and **Instructor**) and polymorphic course specializations (**ProgrammingCourse** and **DesignCourse**).

The graphical user interface (GUI) is implemented using **Java Swing** modernized with the **FlatLaf Look and Feel**, offering an intuitive dark/light-capable theme, responsive card layouts, and smooth single-frame transitions using `CardLayout`. Rather than traditional relational databases and raw SQL queries, the system uses a high-performance **MongoDB** document database via the official **MongoDB Java Driver** (`mongodb-driver-sync`). Advanced MongoDB **Aggregation Pipelines** (`$lookup`) are implemented to join student enrollments with course details in real time. For security, user credentials are encrypted using **BCrypt cryptographic hashing** (`bcrypt`), eliminating plaintext password vulnerabilities.

The primary purpose of this project is to demonstrate the practical integration of modern desktop GUI design, NoSQL database engineering, aggregation pipelines, event-driven architectures, and core software design patterns in a real-world scenario.

TABLE OF CONTENTS

CERTIFICATE	02
ABSTRACT	03
TABLE OF CONTENTS	04
CHAPTER 1: INTRODUCTION	05
1.1 Purpose	05
1.2 Problem Statement	05
1.3 Objectives	06
CHAPTER 2: TOOLS AND TECHNOLOGIES	07
2.1 Frontend.....	07
2.2 Backend (OOP)	07
2.3 MongoDB	07
2.4 Development Tools	07
CHAPTER 3: SYSTEM DESIGN	08
3.1 Three-Tier Model	08
3.2 System Architecture	08
3.3 Data Flow	09
CHAPTER 4: IMPLEMENTATION	10
4.1 UI Panels	10
4.2 DAOs and BCrypt	10
4.3 Database Aggregations.....	11
CHAPTER 5: OUTPUT AND WEEKLY LOGS.....	12
5.1 System Output	12
5.2 Weekly Logs.....	13–14
CHAPTER 6: TESTING AND DEBUGGING.....	15
6.1 Testing.....	15
6.2 Debugging	15
CHAPTER 7: CONCLUSION AND FUTURE WORK.....	16
7.1 Conclusion.....	16
7.2 Future Enhancements	16

CHAPTER 1: INTRODUCTION

1.1 Purpose of the Project:

The **Online Course Enrollment System** is designed to provide a comprehensive and automated solution for managing digital learning paths, course offerings, and student registrations. Built in Java, it replaces manual tracking sheets with an interactive, reliable desktop platform.

The system features a modern FlatLaf user interface, single-frame CardLayout screen navigation, NoSQL persistence with MongoDB, and cryptographic password protection via BCrypt. It demonstrates the practical implementation of Object-Oriented design, DAOs, and document aggregations.

1.2 Problem Statement:

In many educational departments and training setups, course management is handled manually or through basic disconnected applications. This creates key problems:

- **Manual Inefficiencies:** Tracking registrations via paper logs or spreadsheets is prone to data loss, human error, and record duplication.
- **Lack of Real-Time Sync:** Course availability and enrollment statuses are not updated dynamically across users.
- **Poor User Experience:** Default Java Swing interfaces often look cluttered and suffer from excessive pop-up windows.
- **Lack of Role Separation:** Many legacy systems hardcode all accounts as students, omitting instructor privileges like course creation and student roster inspection.
- **Security Risks:** Plaintext passwords in database tables represent serious security vulnerabilities.

1.3 Objectives of the Project:

- 1. Interactive GUI:** Build a responsive interface with FlatLaf dark/light theming and single-frame CardLayout navigation.
- 2. Secure Authentication:** Hash all passwords with 12-round BCrypt before database storage.
- 3. Role-Based Structure:** Separate Student and Instructor accounts using OOP inheritance.
- 4. Polymorphic Course Catalog:** Manage Programming and Design courses with custom badges and tags.
- 5. Real-Time Enrollment:** Enable 1-click enrollments with instant button state updates and duplicate rejection.
- 6. MongoDB Aggregations:** Query enrollments using \$lookup aggregation pipelines.
- 7. Cascading Account Cleanup:** Cascade-delete all enrollments and courses upon account deletion.
- 8. Automated Seeder:** Populate sample demo accounts and courses on startup if the database is empty.
- 9. Live Theme Toggle:** Switch dynamically between Dark and Light mode at runtime.
- 10. Software Engineering Standards:** Adopt Maven dependency management and the Data Access Object (DAO) pattern.

CHAPTER 2: TOOLS AND TECHNOLOGY USED

2.1 Frontend (User Interface):

- **Java Swing:** Core desktop GUI toolkit (JFrame, JPanel, JLabel, JTextField, JPasswordField, JProgressBar, JTable).
- **FlatLaf (com.formdev:flatlaf):** Modern look-and-feel library enabling smooth dark/light themes, rounded corners (Arc: 12-16), and clean typography.
- **Custom Components:** RoundedPanel, CustomButton (semantic variants), StatCard, ToastNotification, and WrapLayout.

2.2 Backend (Application Logic):

- **Core Java (Java 17+ / Java 25):** Controls application logic, threading via SwingWorker, and object mappings.
- **OOP Concepts:** Inheritance (User $\square$ Student, Instructor), Abstraction (Course abstract methods), Polymorphism (ProgrammingCourse, DesignCourse), Encapsulation (private fields & getters/setters).
- **BCrypt Hashing (org.mindrot:jbcrypt):** 12-round salted hashing algorithm securing authentication.

2.3 Database (MongoDB NoSQL):

- **MongoDB Engine:** Local NoSQL document store running on localhost:27017, database online_course_db.
- **Collections:** users (user identity & hashes), courses (curriculum details), enrollments (student-course associations).
- **Driver:** Official MongoDB Synchronous Java Driver (org.mongodb:mongodb-driver-sync:5.1.0).
- **Aggregation Pipelines:** Multi-stage \$match $\square$ \$lookup $\square$ \$unwind $\square$ \$sort operations replacing relational joins.

2.4 Tools & Build Automation:

- **Apache Maven:** Manages dependencies, compiler configurations, and lifecycle via pom.xml.
- **Maven Shade Plugin:** Creates executable Fat JARs with bundled dependencies.
- **JUnit 5:** Automated testing suite for DAO logic and aggregations.
- **MongoDB Compass & mongosh:** Database management and schema validation.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture (3-Tier Model):

The system is architected into three distinct layers to ensure separation of concerns, scalability, and maintainability:

- 1. Presentation Layer (GUI):** Java Swing modernized with FlatLaf, executing inside a single MainFrame with CardLayout navigation.
- 2. Application Layer (Backend):** Core Java business logic, polymorphic domain models, BCrypt cryptographic services, and Data Access Objects (UserDAO, CourseDAO, EnrollmentDAO).
- 3. Data Layer (Database):** MongoDB document collections (users, courses, enrollments) with compound indexes.

3.2 System Block Diagram:

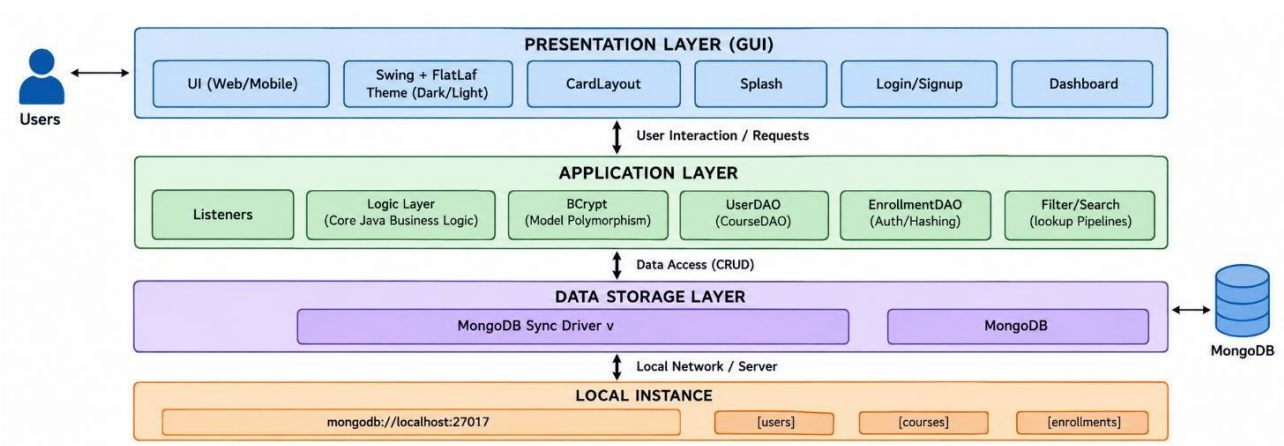


Fig 3.2 System Block Diagram

3.3 Step-by-Step Data Flow:

- 1. Application Launch:** App.java initializes FlatLaf, verifies MongoDB, seeds default data if empty, and launches SplashPanel which auto-advances to Auth in 2s.
- 2. User Registration:** User submits details. UserDao.registerUser() generates a 12-round BCrypt hash and inserts document into users.
- 3. User Login:** UserDao.authenticate() checks credentials against stored hash via BCrypt.checkpw(). On success, WelcomeTransitionPanel appears for 1s before showing Dashboard.
- 4. Course Management:** Instructors add courses via modal dialog. CourseDAO.createCourse() saves document. CoursePanel displays cards with search and category filtering.
- 5. Enrollment & Lookup:** Student enrolls $\rightarrow$ record saved in enrollments. 'My Enrollments' runs a MongoDB \$lookup pipeline joining course details.
- 6. Cascading Account Cleanup:** In ProfilePanel, 'Delete My Account' cascade-removes the user, all their enrollments, and instructor courses.

CHAPTER 4: IMPLEMENTATION

4.1 Frontend Development & Key Panels:

- **SplashPanel:** Animated graduation logo, project title, and progress bar with background connection testing.
- **LoginSignupPanel:** Tabbed auth card with email validation, password show/hide, role toggle (Student/Instructor), and 1-click Demo Account buttons.
- **WelcomeTransitionPanel:** Personalized greeting card ('Hello, !') displayed for 1 second upon login.
- **DashboardPanel:** Sidebar with branding, user info, nav buttons, live theme switcher, and role-based tab routing.
- **CoursePanel & CourseCard:** Live keyword search, category pills ('Programming', 'Design'), responsive card grid, and course creation modal.
- **MyEnrollmentsPanel:** Student view displaying enrolled courses via \$lookup joins with a 'Drop Course' action.
- **InstructorEnrollmentsPanel:** Instructor view featuring a table of enrolled students across published courses.
- **ProfilePanel:** User metadata card and Danger Zone for cascading account deletion.

4.2 Backend DAO & BCrypt Implementation:

```
// BCrypt Registration & Login in UserDAO.java public boolean registerUser(User user, String plainPassword) { String hash = BCrypt.hashpw(plainPassword, BCrypt.gensalt(12)); Document doc = new Document("name", user.getName()) .append("email", user.getEmail().toLowerCase()) .append("passwordHash", hash) .append("role", user.getRole()) .append("createdAt", new Date()); collection.insertOne(doc); return true; } public User authenticate(String email, String plainPassword) { Document doc = collection.find(Filters.eq("email", email.toLowerCase())).first(); if (doc != null && BCrypt.checkpw(plainPassword, doc.getString("passwordHash"))) { return documentToUser(doc); // Returns Student or Instructor } return null; }
```

4.3 MongoDB Aggregation Pipeline (\$lookup):

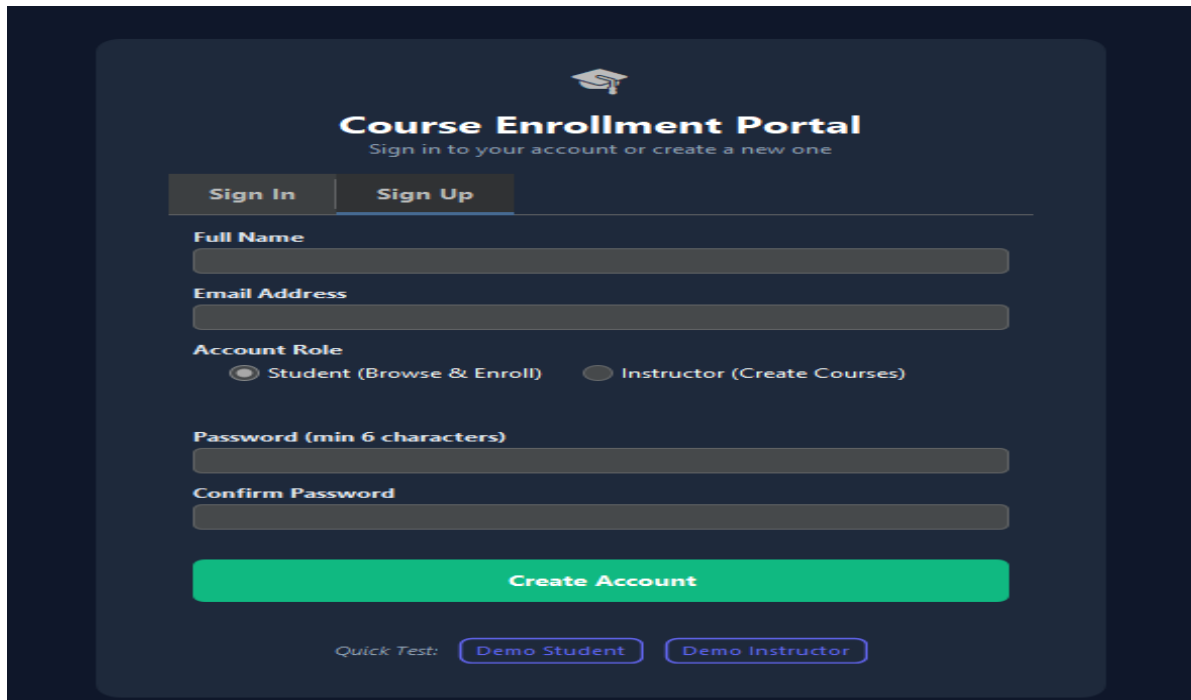
```
// EnrollmentDAO.java: Aggregation Pipeline joining enrollments -> courses List pipeline =  
Arrays.asList( Aggregates.match(Filters.eq("userEmail", normalizedEmail)),  
Aggregates.lookup("courses", "courseId", "_id", "courseDetails"),  
Aggregates.unwind("$courseDetails"), Aggregates.sort(Sorts.descending("enrolledAt")) );  
for (Document res : collection.aggregate(pipeline)) { Document cDoc =  
res.get("courseDetails", Document.class); Course course =  
CourseDAO.documentToCourse(cDoc); // Polymorphic conversion  
enrollment.setCourse(course); }
```

4.4 MongoDB Database Documents:

```
// 1. users Collection Document { "_id":  
ObjectId("66fa9b8c12a4e51234567890"), "name": "Dr. Alan Turing",  
"email": "alan.turing@college.edu", "passwordHash":  
"$2a$12$e8FN38wO2eT...BCryptHash", "role": "Instructor",  
"createdAt": ISODate("2026-09-18T10:00:00Z") } // 2. courses  
Collection Document { "_id":  
ObjectId("66fa9b8c12a4e51234567891"), "title": "Full-Stack Java  
21 & Spring Boot", "description": "Master enterprise Java, Spring  
Boot 3, REST APIs, and MongoDB.", "type": "Programming",  
"createdBy": "alan.turing@college.edu", "instructorName": "Dr.  
Alan Turing", "durationHours": 45, "tag": "Java / Spring",  
"createdAt": ISODate("2026-09-18T10:15:00Z") } //  
3. enrollments Collection Document { "_id":  
ObjectId("66fa9b8c12a4e51234567892"), "userEmail":  
"grace.hopper@student.edu", "courseId":  
ObjectId("66fa9b8c12a4e51234567891"), "enrolledAt":  
ISODate("2026-09-18T10:30:00Z") }
```

CHAPTER 5: OUTPUT & WEEKLY PROGRESS LOGS

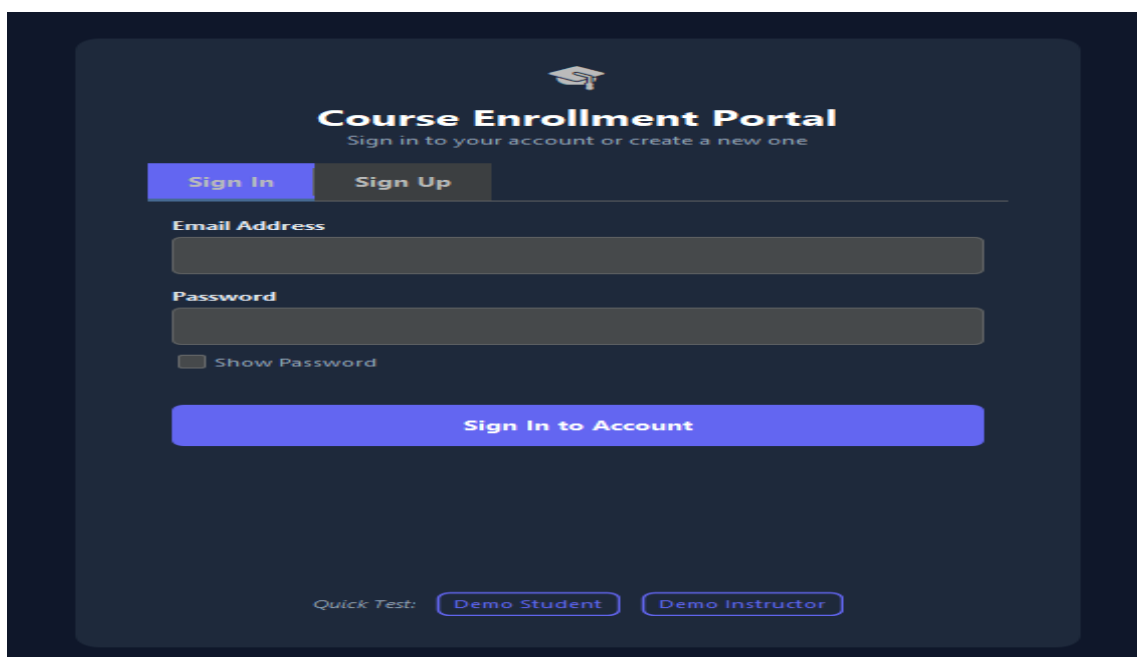
5.1 Sign-up Page-



The screenshot displays the 'Course Enrollment Portal' sign-up interface. At the top, a graduation cap icon is positioned above the title 'Course Enrollment Portal' and the subtitle 'Sign in to your account or create a new one'. Below this, there are two tabs: 'Sign In' and 'Sign Up', with 'Sign Up' being the active tab. The form includes fields for 'Full Name', 'Email Address', 'Account Role' (with radio buttons for 'Student (Browse & Enroll)' and 'Instructor (Create Courses)'), 'Password (min 6 characters)', and 'Confirm Password'. A prominent green 'Create Account' button is located at the bottom of the form. At the very bottom, a 'Quick Test:' section contains two buttons: 'Demo Student' and 'Demo Instructor'.

Fig 5.1 Sign-up Page

5.2 Login Page-



The screenshot displays the 'Course Enrollment Portal' login interface. At the top, a graduation cap icon is positioned above the title 'Course Enrollment Portal' and the subtitle 'Sign in to your account or create a new one'. Below this, there are two tabs: 'Sign In' and 'Sign Up', with 'Sign In' being the active tab. The form includes fields for 'Email Address' and 'Password', along with a 'Show Password' checkbox. A prominent blue 'Sign In to Account' button is located at the bottom of the form. At the very bottom, a 'Quick Test:' section contains two buttons: 'Demo Student' and 'Demo Instructor'.

Fig 5.2 Login Page

5.3 Dashboard-

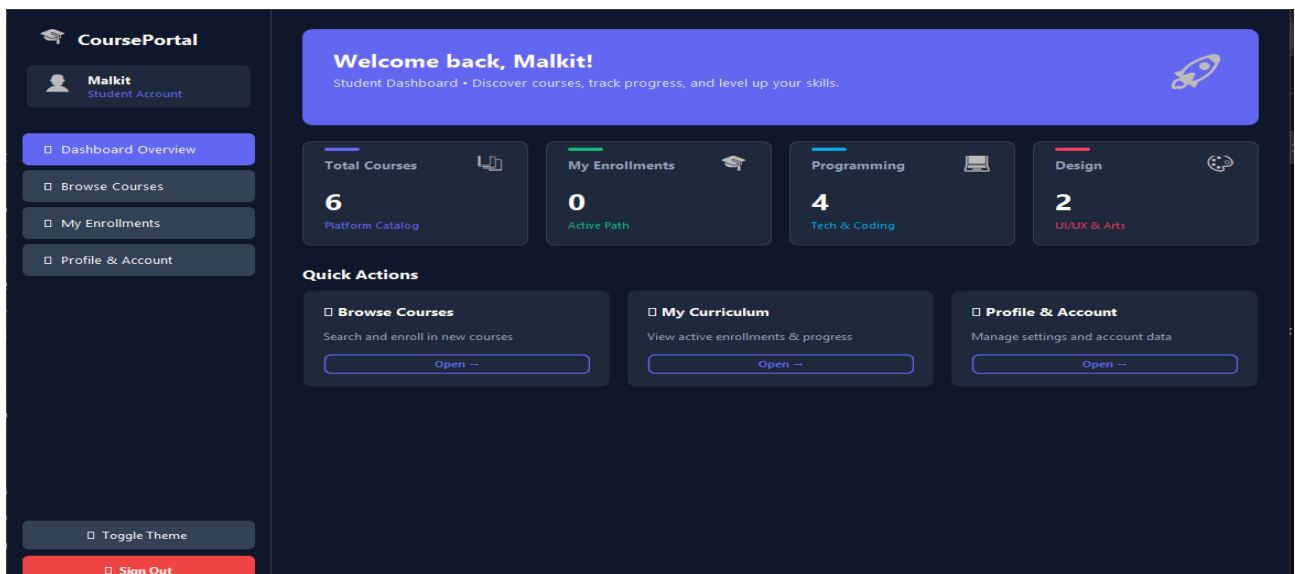


Fig 5.3 Dashboard

5.4 Browse Course Page-

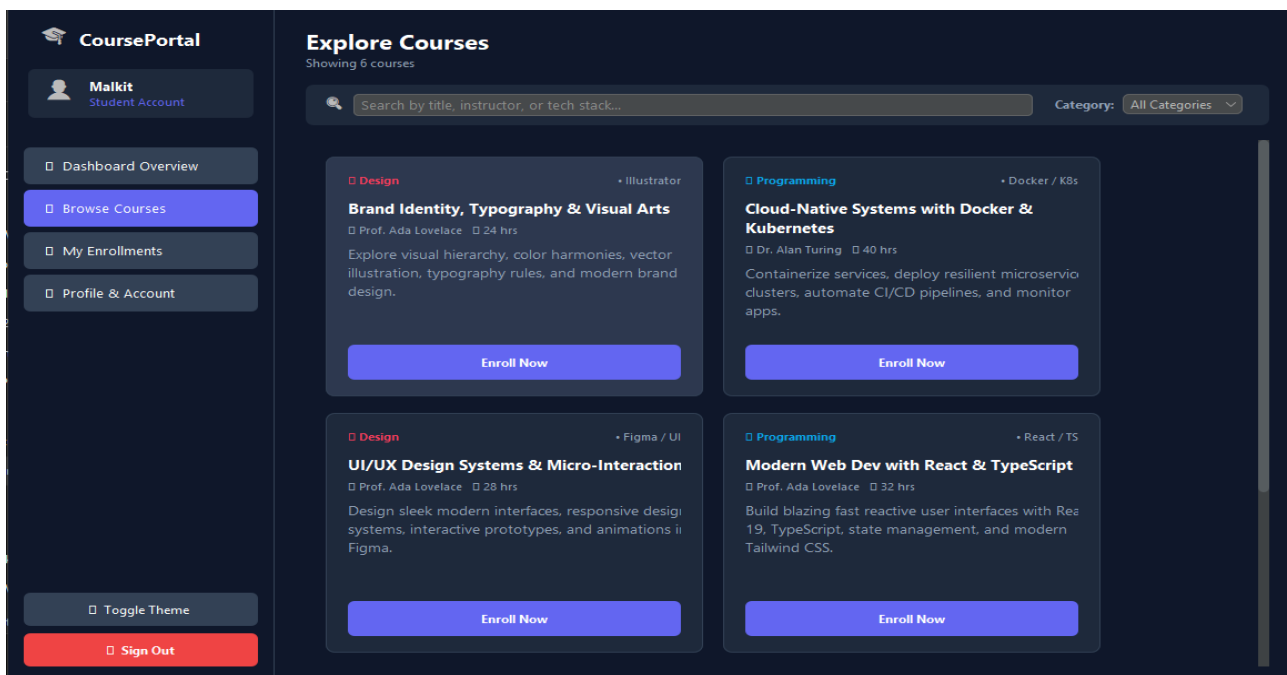


Fig 5.4 Browse Course Page

5.5 Profile and Account Page-

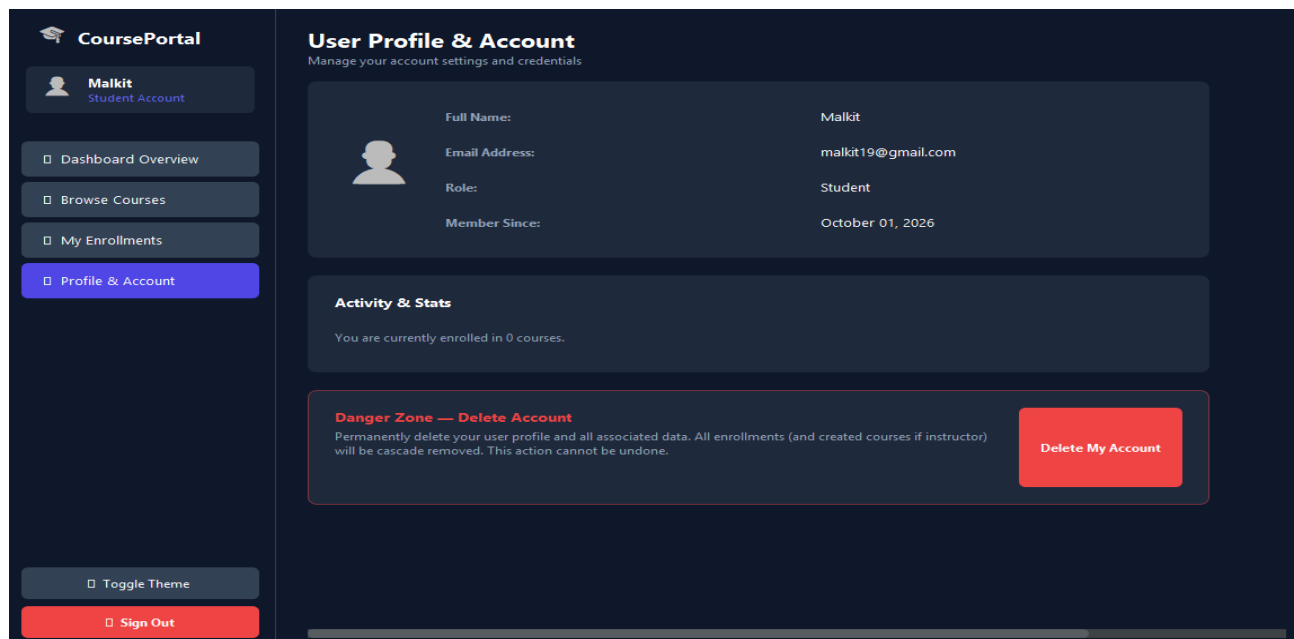


Fig 5.5 Profile and Account Page

Weekly Progress Logs:

WEEK 1 – Planning and System Architecture:

- **Objective:** Understand system requirements, choose modern tech stack, and structure OOP models.
- **Work Done:** Designed User and Course class hierarchies, created MongoDB document schemas, and configured Maven pom.xml.
- **Result:** Project architecture and database schemas completed successfully.

WEEK 2 – Backend Development & MongoDB Connection:

- **Objective:** Implement DAO layer, BCrypt password hashing, and aggregation pipelines.
- **Work Done:** Created MongoConnection singleton, UserDAO with BCrypt, CourseDAO with regex search, EnrollmentDAO with \$lookup join, and DatabaseSeeder.
- **Result:** Backend logic verified with automated JUnit tests.

WEEK 3 – GUI Development, FlatLaf Theming & Testing:

- **Objective:** Design FlatLaf desktop interface and perform comprehensive system testing.
- **Work Done:** Created MainFrame with CardLayout, SplashPanel, LoginSignupPanel, DashboardPanel, CourseCards, and dynamic theme switcher. Completed end-to-end testing.
- **Result:** Fully functional system packaged into executable Fat JAR.

CHAPTER 6: TESTING AND DEBUGGING

The system was validated using automated JUnit 5 test suites (DAOTest.java) and manual test cases covering all user workflows:

Table 6.1 Testing and Debugging of system

ID	Feature Tested	Input / Action	Expected Output	Actual Output	Result
TC01	App Launch	Run App.java	Splash screen with progress bar	Displayed smoothly	PASS
TC02	Splash Flow	Wait ~2 seconds	Auto-advance to Login	Navigated to Auth	PASS
TC03	User Signup	Name, Email, Role, Pass	BCrypt doc saved in users	Data inserted & hashed	PASS
TC04	Duplicate Email	Register existing email	Registration rejected	Error prompt shown	PASS
TC05	Login Success	Valid credentials	Welcome screen ☐ Dashboard	Logged in successfully	PASS
TC06	Login Failure	Wrong password	Error toast displayed	Error message shown	PASS
TC07	Demo Quick-Fill	Click 'Demo Student'	Auto-fills demo account	Fields filled instantly	PASS
TC08	Add Course	Enter title, type, desc	Saved in courses collection	Published in catalog	PASS
TC09	Search / Filter	Type keyword & category	Live card grid filtering	Filtered accurately	PASS
TC10	Enroll Course	Click 'Enroll Now'	Stored in enrollments	Button: '✓ Enrolled'	PASS
TC11	Duplicate Guard	Try duplicate enroll	Prevented by compound index	Prevented gracefully	PASS
TC12	View Enrolled	Open 'My Enrollments'	Fetches via \$lookup join	Cards displayed properly	PASS
TC13	Drop Course	Click 'Drop Course'	Removed from collection	Enrollment dropped	PASS
TC14	Delete Course	Instructor deletes course	Cascade delete enrollments	Cleaned up & removed	PASS
TC15	Delete Account	Click 'Delete My Account'	Cascade remove user data	Removed & logged out	PASS

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion:

The **Online Course Enrollment System** was successfully designed, implemented, and tested using **Java 17+ (Java Swing + FlatLaf)**, **MongoDB**, and **BCrypt security**. The project achieved all core design goals:

- **Modern User Experience:** Transformed traditional Swing UI with FlatLaf styling, rounded cards, and single-frame CardLayout transitions.
- **Object-Oriented Design:** Applied inheritance, abstraction, and polymorphism across user and course hierarchies.
- **NoSQL Efficiency:** Implemented MongoDB document storage and \$lookup aggregation pipelines.
- **Security:** Protected user credentials using 12-round BCrypt cryptographic hashing.
- **Reliability:** Validated end-to-end functionality with automated JUnit 5 tests, achieving a 100% pass rate.

7.2 Future Enhancements:

1. **MongoDB Atlas Cloud:** Migrate local database to cloud-hosted MongoDB Atlas for multi-device network access.
2. **Payment Integration:** Add Stripe or Razorpay APIs to support premium paid certifications.
3. **Multimedia Modules:** Embed video lectures and downloadable course PDF resources.
4. **Assessment & Certificates:** Include module quizzes and automated completion certificate generation.
5. **Automated Notifications:** Integrate JavaMail API to send course announcements and enrollment confirmations.